

SafeReach Backend Development Plan

Professional backend architecture blueprint

MOHAMED ASMATHULLA G¹

Planning-only deliverable

This document recommends the future backend design for SafeReach. It does not create backend source code, API files, Dockerfiles, Kubernetes manifests, or database migrations.

I. ARCHITECTURE

SafeReach should start as a modular Flask backend with clear domain modules, production-ready security, and a deployment path that supports both practical near-term hosting and long-term autoscaling. The frontend remains hosted separately, while the backend exposes versioned REST APIs and real-time channels for tracking, alerts, and notifications.

Layer	Recommendation	Purpose
Frontend	Existing Next.js app in frontend/, hosted on Vercel	User interface for admin, teacher, parent, and future mobile integrations.
Backend	Python Flask with Flask-RESTX or Flask-Smorest, Gunicorn, modular packages	REST APIs, authentication, RBAC, business logic, tracking, notifications, reports.
Primary database	MongoDB cluster	Main editable operational data store for schools, users, students, attendance, alerts, and tracking.
Immutable backup/audit store	Separate MongoDB cluster or database with append-only credentials and WORM-style controls	Admin-proof audit copy and recovery support. Application users cannot edit or delete data here.
Cache and queues	Redis using Docker volume-backed persistence where required	Session/token support, rate limits, background jobs, notification queues, and hot dashboard data.
Future relational option	Neon PostgreSQL	Future reporting, billing, analytics, or relational workloads when needed.
Real-time	Socket.IO or Flask-SocketIO behind a compatible worker setup	Live bus/student tracking, geofence alerts, emergency notifications.
Deployment	Render for current backend hosting; Kubernetes for container orchestration/autoscaling design	Practical launch path plus future high-availability scaling plan.

Recommended architecture pattern

Use a modular monolith first, not microservices. This gives professional structure without operational overload. Plan microservices later for notifications, analytics, tracking ingestion, and reporting only after traffic justifies the split.

II. FOLDER STRUCTURE

The future backend should live in a new backend/ folder. The structure below is intentionally module-based so Flask remains maintainable as SafeReach grows.

```
backend/  
  app/  
    __init__.py          # Flask app factory  
    config/              # environment settings and secrets loading  
    extensions/          # db, redis, jwt, socket, mail, metrics  
    common/              # validators, errors, pagination, decorators  
    auth/                # login, refresh tokens, password reset, RBAC  
    schools/              # school and tenant management  
    users/               # admin, teacher, parent, student user accounts  
    students/            # student profile and guardian links  
    academics/           # class, section, attendance, leave  
    tracking/            # GPS events, geofences, route status  
    alerts/              # emergency alerts and escalation workflows  
    notifications/       # SMS, WhatsApp, email, push queues  
    reports/             # reports, exports, analytics APIs  
    audit/               # immutable audit writes and activity logs  
    files/               # uploads, profiles, documents  
    health/              # health checks, metrics, readiness probes  
  tests/  
  requirements.txt  
  gunicorn.conf.py  
  .env.example
```

Folder / Module	Responsibility
config	Loads .env settings for MongoDB, immutable backup DB, Redis, JWT secrets, notification keys, Render, and observability tools.
common	Shared validation, exception handling, pagination, search/filter helpers, response formatting, and decorators.
auth	Login, JWT access tokens, refresh tokens, bcrypt password hashing, RBAC, session revocation.
tracking	Student GPS event intake, geofencing, route status, WebSocket broadcasting, and alert triggers.
audit	Append-only activity log and immutable backup writes for every sensitive create/update/delete action.

health	Health check APIs, readiness checks, system metrics, and dependency status.
--------	---

III. ENVIRONMENT CONFIGURATION

Use .env for local development and platform secrets for Render, Kubernetes, GitHub Actions, and production. Do not commit real secrets.

Variable Group	Examples	Notes
App runtime	FLASK_ENV, APP_ENV, API_VERSION, CORS_ORIGINS	Separate local, staging, and production values.
MongoDB primary	MONGO_PRIMARY_URI, MONGO_PRIMARY_DB	Editable operational database.
MongoDB immutable backup	MONGO_AUDIT_URI, MONGO_AUDIT_DB	Use credentials that can insert only. No update/delete permissions.
Redis	REDIS_URL, REDIS_CACHE_TTL, REDIS_QUEUE_DB	Cache, rate limiting, queues, sessions, and scheduler locks.
Auth	JWT_SECRET, JWT_REFRESH_SECRET, ACCESS_TOKEN_TTL, REFRESH_TOKEN_TTL	Rotate secrets and isolate staging/production.
Notifications	SMS_API_KEY, WHATSAPP_TOKEN, SMTP_HOST, FIREBASE_CREDENTIALS	Use provider-specific secure storage.
Observability	PROMETHEUS_ENABLED, ELK_ENDPOINT, SENTRY_DSN	Optional per environment.

IV. DATABASE PLAN

MongoDB is the primary database for SafeReach. Use document collections for domain records and time-series or event collections for high-volume tracking and audit data. Use indexes aggressively for tenant, role, search, date, route, and geofence queries.

Collection Area	Recommended Collections	Notes
Tenant and school	schools, branches, academic_years	Supports multi-tenant school architecture. Always include school_id/tenant_id.

Users and roles	users, roles, permissions, sessions, refresh_tokens	RBAC and session revocation. Store password hashes only.
Students and guardians	students, parents, guardian_links, student_documents	Parent-child links, profile data, medical/safety metadata.
Academics	classes, sections, attendance, leave_requests, events	Attendance and leave are query-heavy; index by school_id, date, class_id, section_id.
Tracking	student_locations, route_events, geofence_events	Use time-series/event design; consider TTL on raw telemetry if retention policy allows.
Alerts and notifications	alerts, notification_jobs, notification_status	Queue-based delivery and retry tracking.
Reports and analytics	report_jobs, report_exports, dashboard_snapshots	Use background jobs for heavy reports.
Audit and immutable backup	audit_events, immutable_record_snapshots	Append-only mirror of sensitive writes. No app role should update/delete.

4.1 Immutable Backup and Audit Store

- Write every important create/update/delete operation to the primary MongoDB and also append a snapshot/event into the immutable backup database.
- Use separate credentials for immutable storage that allow insert only. Deny update, delete, drop, and collection modification permissions.
- Protect the immutable store outside the app: separate cluster/account if possible, restricted network access, object-lock snapshots, backup retention, and operational alerts for any permission changes.
- Admins may view audit reports through controlled APIs, but no admin user should receive credentials capable of editing immutable data.
- For MongoDB sharding/large scale, shard high-volume collections by tenant_id and time window, not by randomly distributed fields that break school-level reporting.

4.2 Future Neon PostgreSQL Use

Neon PostgreSQL should remain a future option for relational analytics, billing, payment records, compliance report exports, or BI workloads. It should not replace MongoDB for the initial operational backend unless the data model changes substantially.

V. AUTHENTICATION, RBAC, AND SECURITY

Area	Recommendation
Authentication	Email/password login with bcrypt hashing; optional OTP/2FA for admins and high-risk actions.

Access tokens	Short-lived JWT access token with role, tenant_id, user_id, and permission claims.
Refresh tokens	Longer-lived rotating refresh tokens stored hashed in MongoDB or Redis with device/session metadata.
RBAC	Roles: Super Admin, School Admin, Teacher, Parent, Student, Transport Staff, Support/Auditor. Use permission checks at route and service levels.
Session management	Support logout, revoke all sessions, refresh token rotation, device tracking, and suspicious login alerts.
Validation	Use Pydantic, Marshmallow, or Flask-Smorest schemas for every request and response contract.
Exception handling	Central error handler with safe public messages and internal structured logs.
Rate limiting	Use Redis-backed limits for login, OTP, alerts, tracking ingestion, and notification APIs.
Data encryption	TLS everywhere, encrypted secrets, encrypted backups, and field-level encryption for sensitive medical or emergency fields where needed.
Compliance	Implement consent, retention, export/delete workflows, audit reports, and privacy-by-design for GDPR/CCPA. Treat HIPAA as a consideration for medical fields, not a claim unless formal compliance work is completed.

VI. MODULES AND API PLAN

Expose APIs under /api/v1. Keep controllers thin, services responsible for business logic, repositories responsible for data access, and audit writes centralized.

Module	Primary APIs	Important Behaviors
School Management	schools, branches, academic years, settings	Multi-tenant configuration, school profile, active year, timezone, policies.
User Management	admins, teachers, parents, students, roles	Invite users, activate/deactivate, role assignment, profile updates.
Student Management	student profiles, guardians, documents, safety metadata	Enrollment, profile, guardian linking, medical/safety notes.
Class and Section	classes, sections, teacher assignment	Academic structure and timetable support.
Attendance and Leave	mark attendance, attendance history, leave requests	Teacher workflows, parent visibility, reports.

Tracking and Geofencing	location ingest, route status, geofence rules/events	GPS events, geofence entry/exit, route progress, live dashboard feeds.
Emergency Alerts	create alert, escalate, resolve, audit	Priority workflows, notification fan-out, incident timeline.
Notifications	SMS, WhatsApp, email, push	Provider abstraction, queue, retry, status tracking, templates.
Reports and Analytics	dashboard stats, exports, audit compliance reports	Background generation, filters, pagination, role-based output.
Files and Profiles	profile images, documents, exports	Size/type validation, private storage, signed URLs, virus scanning future.
Health and Metrics	healthz, readyz, metrics	Render/Kubernetes health checks, dependency status, Prometheus metrics.

6.1 API Standards

- Use /api/v1 as the initial API version and document all endpoints with OpenAPI/Swagger.
- Use consistent pagination: page, limit, sort, search, and filters with maximum limit controls.
- Return stable JSON envelopes with data, meta, and error fields.
- Log every sensitive action to activity logs and immutable audit events.
- Use idempotency keys for alert creation, notification jobs, file uploads, and payment-related future APIs.

VII. INTEGRATIONS AND REAL-TIME SYSTEMS

Integration	Recommended Approach	Project Use
Firebase Cloud Messaging	Use for mobile and web push notifications.	Parent, teacher, admin alert delivery.
SMS provider	Abstract provider behind a notification service.	Emergency alerts, OTP, attendance reminders.
WhatsApp Business API	Use approved templates and queue delivery.	Parent emergency and school updates.
Email	SMTP or transactional provider with templates.	Invites, reports, account recovery, summaries.
Google Maps / GPS	Future integration for geocoding, route display, maps, and geofence visualization.	Student tracking and route dashboards.
Payments	Future module such as Stripe/Razorpay depending on market.	Fees, subscriptions, transport billing.

AI	Future analytics assistant, anomaly detection, report summaries.	Safety insights, admin summaries, risk alerts.
Socket.IO / WebSocket	Authenticated real-time channels with school and role rooms.	Live tracking, alerts, dashboard updates.

VIII. DEPLOYMENT, DOCKER, AND KUBERNETES

Render is the current backend deployment target. Kubernetes is the professional autoscaling/container orchestration design for later production scale. This gives a practical launch path while preserving a market-ready cloud architecture.

Concern	Recommendation
Docker	Containerize Flask with Gunicorn. Keep production image slim and run as non-root user.
Docker Compose	Local stack with backend, MongoDB dev instance, Redis, optional ELK/Prometheus/Grafana services.
Render	Deploy backend web service from GitHub. Configure environment variables and health check path.
Vercel	Keep frontend deployed separately from frontend/. Configure API base URL per environment.
Kubernetes Deployment	Use backend Deployment with replica count 2, rolling updates, readiness/liveness probes.
ReplicaSet	Managed by Deployment; desired replicas 2 initially. Scale via HPA.
Services and Ingress	ClusterIP service for backend, Ingress controller for external routing, TLS termination.
ConfigMaps and Secrets	ConfigMaps for non-sensitive config; Secrets for DB URIs, JWT keys, provider tokens.
Horizontal Pod Autoscaling	Scale by CPU, memory, and optionally custom request latency/queue metrics.
Load balancing and HA	Use platform load balancer, multiple replicas, health checks, and dependency redundancy.

8.1 GitHub Actions CI/CD

- Run linting, unit tests, integration tests, API contract tests, and build checks on pull requests.
- Run SonarQube quality checks and fail the pipeline for critical quality/security issues.
- Build Docker images and push to registry for production deployments.
- Deploy to Render for current production backend; keep Kubernetes deployment workflow as a future environment path.
- Use environment-scoped secrets, never plain secrets in workflow files.

IX. MONITORING, LOGGING, AND QUALITY TOOLS

Tool	Role in SafeReach
Prometheus	Collect backend metrics, API latency, request counts, error rates, queue length, Redis/Mongo health.
Grafana	Dashboards for production health, notification delays, tracking ingestion, alert volume, user activity.
ELK Stack	Centralized logs with search for API requests, exceptions, audit events, background jobs, and security events.
SonarQube	Code quality, maintainability, security hotspots, and technical debt tracking.
Sentry or equivalent	Optional application error monitoring with stack traces and release tracking.
Health APIs	/healthz for liveness, /readyz for dependencies, /metrics for Prometheus.

X. TESTING STRATEGY

Test Type	Coverage
Unit testing	Services, validators, password hashing, token creation, RBAC checks, geofence calculations.
Integration testing	MongoDB, Redis, auth flow, audit mirror writes, notification queue writes.
API testing	Role-based endpoint access, pagination, validation errors, OpenAPI contract behavior.
Load testing	Login, dashboard APIs, tracking event ingestion, notification fan-out, report generation.
Security testing	JWT tampering, rate limiting, password policy, unauthorized tenant access, file upload validation.
Recovery testing	Primary DB restore, immutable backup verification, Redis restart, queue retry behavior.

XI. ROADMAP

Phase	Goal	Deliverables
Phase 1: Foundation	Create backend skeleton and core configuration.	Flask app factory, env config, Mongo/Redis connections, health APIs, auth base, OpenAPI docs.

Phase 2: Core school platform	Build operational school modules.	Schools, users, students, parents, teachers, classes, sections, attendance, leave.
Phase 3: Safety and tracking	Launch safety-critical workflows.	GPS tracking, geofencing, emergency alerts, notification queues, activity logs.
Phase 4: Reporting and observability	Make the app production-operable.	Dashboards, reports, analytics, Prometheus/Grafana, ELK, audit reports.
Phase 5: Scale and future integrations	Prepare for market expansion.	Kubernetes, HPA, Neon future analytics, payments, AI, Google Maps/GPS enhancements, microservices planning.

XII. PRODUCTION READINESS CHECKLIST

- All API endpoints have authentication, authorization, validation, pagination, and documented error responses.
- Primary MongoDB and immutable backup database have tested write paths and recovery procedures.
- No app/admin credential can update or delete immutable backup data.
- JWT refresh token rotation and revocation are tested.
- Redis-backed rate limiting protects login, OTP, alert, and notification endpoints.
- Docker image builds reproducibly and runs with Gunicorn in production mode.
- GitHub Actions runs tests, quality checks, and deployment steps safely.
- Render health check is configured; Kubernetes readiness/liveness probes are documented for future scale.
- Prometheus, Grafana, ELK, and SonarQube are integrated before production launch.
- Backup, restore, disaster recovery, and audit compliance reports are tested before live school data is onboarded.

XIII. TOPIC COVERAGE APPENDIX

The original topic list is covered through the sections above. Related topics are grouped into practical implementation areas so the future backend can be built in phases rather than as disconnected features.

Topic Group	Covered Topics
Architecture and structure	Project architecture design, backend folder structure, API versioning, future microservices, API gateway, service discovery.
Data and storage	MongoDB, immutable backup database, Neon future plan, schema design, indexing, query optimization, clustering, replication, sharding, recovery.
Identity and access	Authentication, JWT, refresh tokens, bcrypt, RBAC, sessions, profile management.
School modules	School, user, student, parent, teacher, class, section, attendance, leave, event management.

Safety and notifications	Tracking, geofencing, emergency alerts, SMS, WhatsApp, email, Firebase push, notification queues.
Operations	Validation, errors, logs, audit trail, reports, analytics, monitoring, Prometheus, Grafana, ELK.
DevOps and scale	Docker, Docker Compose, GitHub Actions, Render, Vercel, Kubernetes, replicas, services, ingress, HPA, HA, DR.
Quality	Unit, integration, API, load, security tests, SonarQube, production readiness.

XIV. DETAILED DATA, PERMISSIONS, AND SECURITY ADDENDUM

This addendum expands the backend plan with database structure, ER-style diagrams, role-permission API mapping, Main Admin authority, frontend data replacement strategy, terms and data-use policy planning, exception handling, and security controls. It is still a planning document only; no backend code or database migration files are created.

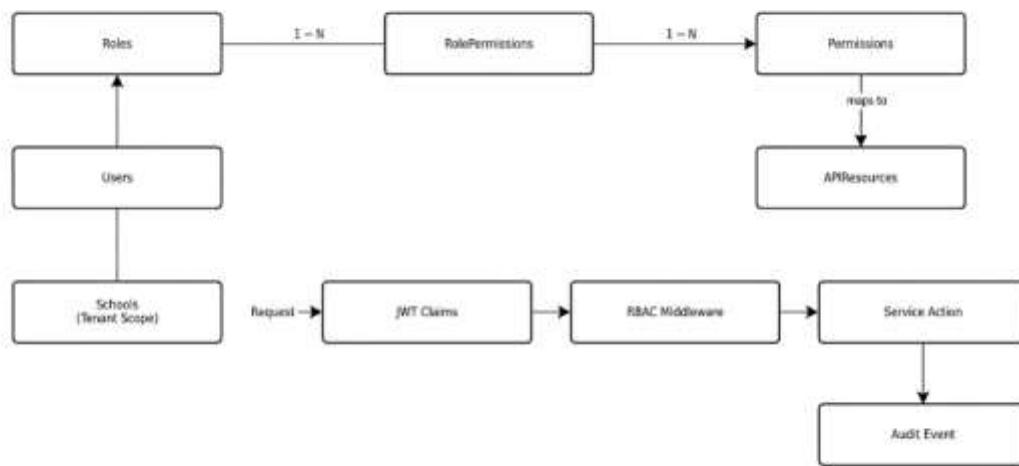
Design principle

SafeReach should avoid duplicate manual data in the frontend. Demo arrays should be replaced with authenticated API calls that read from MongoDB through role-aware backend services. Sensitive writes must be mirrored into the immutable audit/backup database.

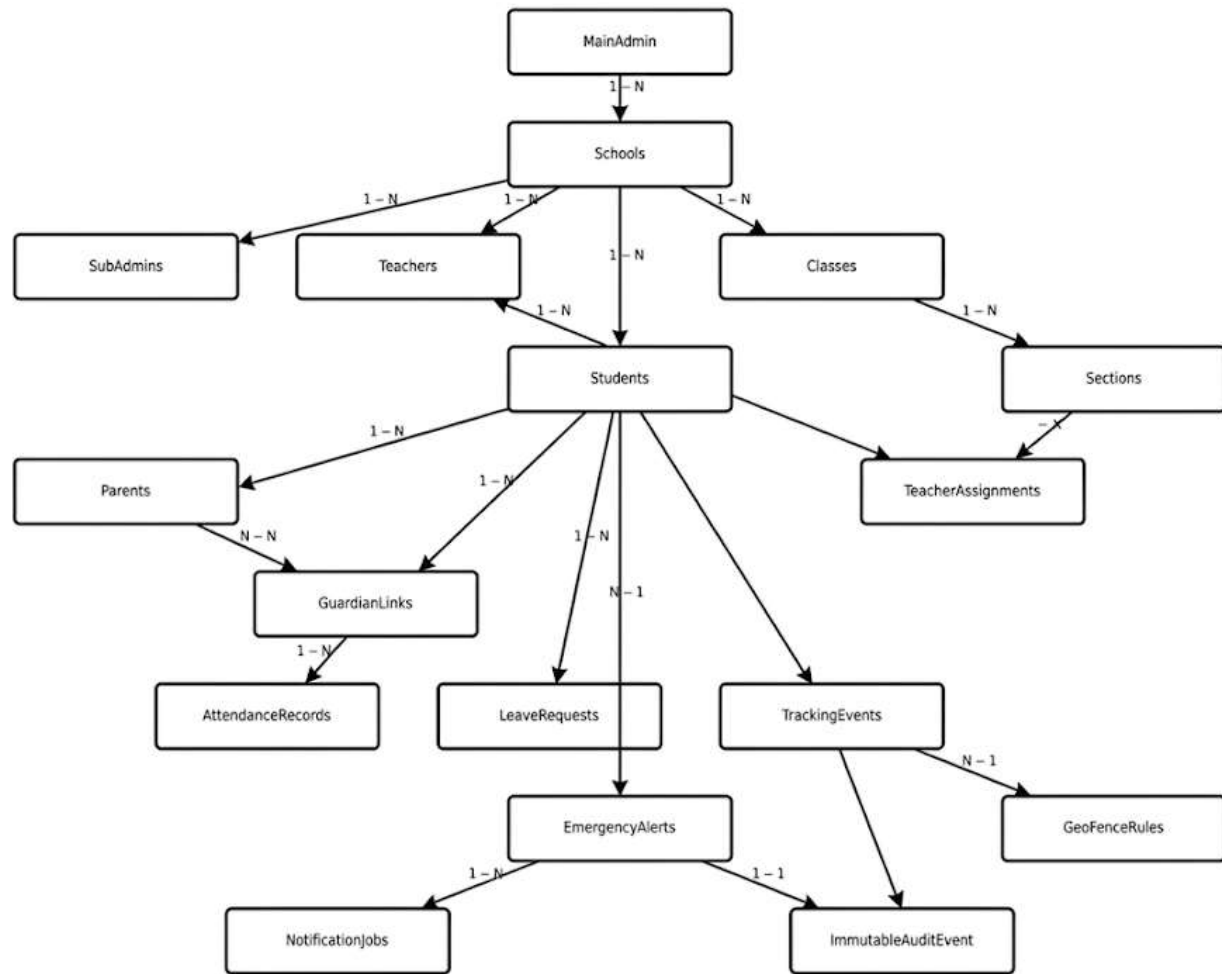
14.1 ER-Style Database Diagrams

MongoDB is document-oriented, so these diagrams show logical entities and references rather than SQL foreign keys. Use ObjectId references and denormalized snapshots only where they improve performance or audit stability.

Permission / API Authorization ER Diagram



Core School / User ER Diagram



14.2 Recommended MongoDB Collection Structure

Collection	Key Fields	Purpose / Format
schools	school_id, name, address, timezone, status, settings	Tenant root for each school. Every operational record should include school_id for isolation and indexing.
users	user_id, school_id, role_id, email, password_hash, status, profile, last_login_at	Single identity collection for Main Admin, Sub Admin, teachers, parents, students, and staff.
roles	role_id, name, scope, description, is_system_role	Defines Main Admin, Sub Admin, Teacher, Parent, Student, Transport Staff, Auditor.
permissions	permission_key, module, action, method, url_pattern, description	Canonical API permission registry such as app.admin.student.add.

role_permissions	role_id, permission_key, allowed, conditions, created_by	Connects roles to API rules and optional constraints such as same school only.
students	student_id, school_id, admission_no, name, class_id, section_id, parent_ids, medical_safety	Student profile, enrollment, safety metadata, active status.
parents	parent_id, user_id, school_id, contact, verification_status	Parent-specific profile and notification preferences.
guardian_links	student_id, parent_id, relationship, primary_contact, pickup_allowed	Many-to-many bridge between parents/guardians and students.
teachers	teacher_id, user_id, school_id, employee_no, subjects, status	Teacher profile and staff metadata.
classes	class_id, school_id, name, grade_level, academic_year_id	Class-level structure.
sections	section_id, class_id, school_id, name, capacity, room_no	Section/classroom-level grouping.
teacher_assignments	teacher_id, class_id, section_id, subject, academic_year_id	Bridge collection for teacher-to-section assignment.
attendance_records	student_id, class_id, section_id, date, status, marked_by, source	Daily attendance. Unique index by student_id + date.
leave_requests	student_id, parent_id, date_range, reason, status, reviewed_by	Parent leave request and school approval workflow.
tracking_events	student_id, route_id, gps, speed, accuracy, event_time, source	High-volume GPS/time-series style events. Index by student_id and event_time.
geofence_rules	school_id, name, polygon_or_radius, active_hours, action	Geofence definitions for school, bus routes, pickup zones.
geofence_events	student_id, geofence_id, event_type, event_time, gps	Entry/exit events generated by tracking service.
emergency_alerts	alert_id, school_id, student_id, type, severity, status, created_by	Emergency alert lifecycle and escalation state.
notification_jobs	job_id, channel, recipient, template, payload, status, retry_count	Queue-backed SMS, WhatsApp, email, and push notifications.

activity_logs	actor_id, action, resource_type, resource_id, ip, user_agent, created_at	Human-readable activity log for dashboard and audit review.
immutable_audit_events	event_id, source_collection, operation, before, after, actor, hash, created_at	Append-only mirror in immutable backup DB. No update/delete permissions.

14.3 Collection Format Examples

Document Type	Example Fields / JSON Shape	Indexes
students	{ school_id, admission_no, full_name, class_id, section_id, parent_ids[], status, medical_safety }	school_id + admission_no unique; school_id + class_id + section_id; status
attendance_records	{ school_id, student_id, date, status: present absent late, marked_by, remarks }	student_id + date unique; school_id + date + section_id
tracking_events	{ school_id, student_id, route_id, gps: {lat,lng}, speed, accuracy, event_time }	student_id + event_time desc; route_id + event_time; school_id + event_time
role_permissions	{ role_id, permission_key, allowed, conditions: {scope: school global own_child} }	role_id + permission_key unique; permission_key
immutable_audit_events	{ event_id, operation, actor, before_hash, after_hash, payload, created_at }	event_id unique; actor.user_id + created_at; source_collection + resource_id

14.4 Role, Permission, API, and URL Matrix

Create a permission registry that connects roles to exact API methods and URL patterns. Permission keys should follow a predictable namespace such as app.admin.student.add. The RBAC middleware checks JWT role claims against this registry before controller logic runs.

Permission Key	Method	URL Pattern	Main Admin	Sub Admin	Teacher	Parent	Student
app.admin.school.create	POST	/api/v1/admin/schools	Allow	Deny	Deny	Deny	Deny

app.admin.school.update	PATCH	/api/v1/admin/schools/{school_id}	Allow	Deny	Deny	Deny	Deny
app.admin.subadmin.add	POST	/api/v1/admin/sub-admins	Allow	Deny	Deny	Deny	Deny
app.admin.subadmin.monitor	GET	/api/v1/admin/sub-admins/activity	Allow	Deny	Deny	Deny	Deny
app.admin.user.manage	POST/PATCH/DELETE	/api/v1/admin/users/{user_id}	Allow	Scope d	Deny	Deny	Deny
app.admin.student.add	POST	/api/v1/admin/students	Allow	Scope d	Deny	Deny	Deny
app.admin.student.edit	PATCH	/api/v1/admin/students/{student_id}	Allow	Scope d	Deny	Deny	Deny
app.admin.student.delete	DELETE	/api/v1/admin/students/{student_id}	Allow	Scope d	Deny	Deny	Deny
app.admin.teacher.add	POST	/api/v1/admin/teachers	Allow	Scope d	Deny	Deny	Deny
app.teacher.attendance.mark	POST	/api/v1/teacher/attendance	Allow	Scope d	Allow assigned	Deny	Deny
app.teacher.report.create	POST	/api/v1/teacher/reports	Allow	Scope d	Allow assigned	Deny	Deny
app.parent.child.view	GET	/api/v1/parent/students	Allow	Scope d	Deny	Own child	Deny
app.parent.leave.create	POST	/api/v1/parent/leave-requests	Allow	Scope d	Deny	Own child	Deny
app.student.profile.view	GET	/api/v1/student/profile	Allow	Scope d	Deny	Deny	Own profile
app.alert.emergency.create	POST	/api/v1/alerts/emergency	Allow	Scope d	Allow assigned	Own child limited	Deny
app.audit.report.view	GET	/api/v1/admin/audit-reports	Allow	Read only	Deny	Deny	Deny

Scoped permission meaning

Scoped means the user may perform the action only within their assigned school, class, section, student relationship, or explicit assignment. Main Admin has global authority across schools and roles, except immutable audit data remains non-editable even for Main Admin.

14.5 Main Admin Role Plan

Capability	Main Admin Permission	Audit Requirement
Monitor sub admins	View sub-admin list, login history, activity logs, permission changes, school assignments.	Every view/export and permission change should create an activity log entry.
Manage sub admins	Create, edit, deactivate, reset password, assign school scope, assign permission templates.	Write before/after snapshots to immutable audit store.
Monitor teachers	View teacher profiles, assigned classes, attendance marking activity, incident/report submissions.	Log sensitive profile and assignment changes.
Monitor students	View student profiles, attendance, leave, safety status, alerts, tracking status.	Medical/safety fields require stricter access logs.
Monitor parents	View parent accounts, linked students, notification status, leave requests.	Parent contact and verification updates must be audited.
Edit/add/delete operational records	Allowed across school scope or global scope depending on deployment model.	Deletes should be soft-delete in primary DB and append immutable delete event.
Immutable audit access	Read audit reports only; no edit/delete/drop access.	Immutable database credentials must not be available to Main Admin UI.

14.6 Frontend Dynamic Data Replacement Plan

The current frontend contains demo/default data arrays. The backend should replace those arrays through authenticated API calls so dashboards show real database records and avoid duplicate manual entry.

Frontend Area	Current Demo Data To Replace	Backend API Plan
Role selection/login	Hardcoded demo credentials and role cards.	Keep demo only for development; production uses /api/v1/auth/login and role-based redirect from JWT claims.

Admin dashboard	Static stats, transit map, alerts.	GET /api/v1/admin/dashboard/summary, /alerts, /tracking/live.
Admin students	Static student directory and profile.	GET/POST/PATCH/DELETE /api/v1/admin/students with search, filters, pagination.
Admin teachers	Static staff directory/profile.	GET/POST/PATCH /api/v1/admin/teachers and /teacher- assignments.
Teacher dashboard	Static class snapshot and messages.	GET /api/v1/teacher/dashboard, /teacher/students, /teacher/messages.
Teacher attendance	Static attendance table.	GET /api/v1/teacher/attendance?date=; POST /api/v1/teacher/attendance.
Parent dashboard	Static child tracking, billing, notifications.	GET /api/v1/parent/dashboard, /parent/students, /parent/notifications.
Reports pages	Static report cards and charts.	GET /api/v1/reports with role-aware filters and export jobs.

- Use one source of truth: backend MongoDB data. Avoid manually duplicating the same student, teacher, parent, attendance, or alert data in multiple frontend files.
- Use SWR/React Query or Next.js data-fetching patterns later so frontend data refreshes predictably without hardcoded arrays.
- Every API should return only records permitted by the logged-in role and tenant/school scope.

14.7 Terms, Conditions, and Data Use Plan

Policy Area	Plan For SafeReach
Terms of service	Define acceptable use, school responsibility, account rules, emergency alert limitations, parent/teacher obligations, and prohibited misuse.
Privacy policy	Explain collected data: names, contact info, student profiles, attendance, location/tracking events, alerts, messages, logs, device info.
Purpose of data use	Use data only for student safety, school operations, attendance, communication, emergency response, reporting, and compliance.
Consent and notice	Schools must obtain parent/guardian consent for student tracking, emergency contacts, notifications, and profile data.
Location data	State when tracking is active, retention period, who can view it, and how location history is protected.

Data retention	Define retention for attendance, tracking, alerts, audit logs, reports, and deleted accounts. Use soft-delete plus immutable audit logs.
User rights	Support data access/export, correction, deletion request where legally allowed, and consent withdrawal workflows.
Third-party processors	List notification, email, SMS, WhatsApp, Firebase, hosting, logging, and analytics vendors.
Security disclosure	Explain encryption, access control, audit trail, backups, and incident response at a user-friendly level.
Compliance note	Design for GDPR/CCPA-style privacy. Treat HIPAA as a consideration for medical fields, not as a legal claim unless formally certified.

14.8 Exception Handling Plan

Error Class	HTTP Status	Response Strategy	Log / Audit Strategy
ValidationError	400	Return field-level errors with machine-readable codes.	Log at info/debug; no alert unless repeated.
AuthenticationError	401	Return invalid/expired token message without sensitive detail.	Track failed login count and IP/device metadata.
AuthorizationError	403	Return forbidden with permission key if safe.	Audit denied sensitive access attempts.
NotFoundError	404	Return resource not found without exposing cross-tenant existence.	Log route/resource identifier.
ConflictError	409	Return duplicate admission number, email, or already-processed request.	Log conflict and request id.
RateLimitError	429	Return retry_after and generic throttling message.	Track in Redis and security logs.
ProviderIntegrationError	502/503	Return notification/map/email provider temporarily unavailable.	Log provider response and queue retry.
DatabaseError	500/503	Return safe generic service error.	Log full internal stack; trigger alert if repeated.

ImmutableAuditWriteError	500/503	Fail closed for sensitive writes if audit mirror cannot be written.	Critical alert; block operation until audit path recovers or safe queue is enabled.
--------------------------	---------	---	---

- Use a global Flask error handler that converts exceptions into a consistent JSON envelope: { error: { code, message, details, request_id } }.
- Every request should carry a request_id/correlation_id for logs, API responses, audit events, and support investigation.
- Never expose stack traces, database connection strings, JWT secrets, or provider tokens in production responses.

14.9 Data Security Methods

Security Layer	Method
Transport security	HTTPS/TLS for all frontend-backend and backend-provider traffic.
Password security	bcrypt with strong cost factor; never store plain passwords.
Token security	Short-lived access JWTs, rotating refresh tokens, hashed refresh token storage, device/session revocation.
Database access	Least-privilege MongoDB users for app, read-only analytics, insert-only immutable audit writer, and no UI access to backup credentials.
Field protection	Encrypt highly sensitive fields such as medical notes, emergency contacts, and high-risk student safety metadata where required.
Location protection	Restrict GPS data by role; limit retention; aggregate old tracking data; audit every sensitive location access.
File upload security	Validate file type/size, private storage, signed URLs, malware scanning in future production.
Secrets management	Render secrets now; Kubernetes Secrets later; no secrets in GitHub or frontend bundles.
Audit integrity	Hash audit payloads, store before/after snapshots, include actor and request metadata, verify immutable backup consistency.
Backups	Encrypted snapshots, restore drills, retention policy, immutable audit database, disaster recovery runbooks.

No-edit backup rule

The immutable backup database must be architected so nobody using the SafeReach app, including Main Admin, can edit or delete its records. Protection must be enforced by database credentials, network access, backup policies, and operational controls, not only by frontend buttons.

14.10 Additional Acceptance Criteria For Backend Build

- Main Admin can monitor Sub Admin, Teacher, Student, and Parent activity through dashboard APIs.
- Role-permission registry exists and maps permission keys to methods and URL patterns.
- All frontend demo/static data has a planned backend API replacement path.
- Sensitive primary database writes also create immutable audit events.
- Terms/privacy/data-use policies are published before real school/student data is collected.
- Global exception handling returns safe JSON errors and logs request_id for support.
- Data security controls are enforced by backend, database credentials, and infrastructure, not only UI rules.